2022 IEEE 13th International Symposium on Power Electronics for Distributed Generation Systems (PEDG) | 978-1-6654-6618-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/PEDG54999.2022.9923317

OCSS: An application to simulate multiple charging stations which uses an Open charge point protocol for communication

Sandeep Yadav Mattepu
Energy Systems and Infrastructures
Fraunhofer IFF
Magdeburg, Germany
ORCID: 0000-0002-4144-3305

Marc Richter
Energy Systems and Infrastructures
Fraunhofer IFF
Magdeburg, Germany
ORCID: 0000-0002-6563-8123

Stephan Balischewski
Energy Systems and Infrastructures
Fraunhofer IFF
Magdeburg, Germany
ORCID: 0000-0001-5935-7221

***Abstract*—Electrification of the transportation sector requires digital tools for managing charging station infrastructure. Therefore, the Open charge point protocol (OCPP) was introduced as a standard communication protocol between a charging station and a central management system to manage such infrastructure effectively. However, developing and especially testing the features described in OCPP can be rather challenging. This study describes the software architecture used to develop OCSS (OCPP charging station simulator), which was driven by the requirements of central management system developers. The benefits of using such a simulator are also extensively discussed. OCSS uses OCPP 1.6 protocol and can perform functions as described in the core and smart charging profiles of OCPP 1.6. We have also discussed the applications of the proposed simulator in this paper.**

***Index Terms*—smart grid, charging infrastructure, OCPP, protocols, charging stations, electric vehicles**

I. Introduction

With the introduction of the Open charge point protocol (OCPP), the communication between a charging station and a central management system is no longer vendor-specific [1] [2]. OCPP proposes several functionalities and specifies how electric vehicles and central management system should function and what messages need to be exchanged to perform a given functionality. The owners of charging stations can easily switch between different central management systems because of the interoperability provided by OCPP; this can make the market of central management system competitive. To be competitive in the market, the developers of central management system software must develop it without any defects. They should be able to handle several edge cases that are not mentioned in the OCPP specification. Although developers of central management system could increase the quality of their software by performing automated testing [3], this form of testing does not provide the flexibility to quickly create new tests or quickly modify existing tests to simulate complex real-world scenarios.

During the development of central management system, developers would need a tool to test their system and evaluate how well their system is working in an actual production environment. Similarly, clients who have charging stations and want to test available central management system software in the market for stress tests based on different scenarios before investing would also need a tool for testing. Finally, researchers who want to develop new load balancing techniques would like to test their methods before deploying them. Therefore, a reliable simulator that can mimic one or more charging stations can be helpful in the scenarios mentioned above. This work explains the software architecture used to create the OCPP charging station simulator (OCSS) that can imitate actual charging stations that use OCPP version 1.6 as communication protocol and some potential use cases of this simulator.

II. Design consideration

A. Object oriented

We have used object-oriented programming paradigm [4] to develop the simulator. In this simulator, a single object represents a single functionality; each object is solely responsible for performing a specific action on the virtual charging station. When a central management system receives a message, a single object (based on the message's contents) is responsible for performing appropriate action on a virtual charging station. For example: In the case of smart charging [5], when central management system sends a "SetChargingProfile" message to virtual charging stations, there is an object in the simulator with the name *SetChargingProfileLogic* that is responsible for handling this message and changing the max number of amperes allowed based on charging station's state. Another example: When we simulate a charging session in a virtual charging station, a "StartCharging" message would be sent to central management system if the virtual charging station is in a valid state. The responsibility of sending the "StartCharging" message would be on the *StartChargingLogic* object.

B. B. Complete abstraction

In the previous section II-A, we have described how each object is responsible for a single functionality of OCPP. The object responsible for performing a function will delegate

Authorized licensed use limited to: FH Salzburg. Downloaded on September 29,2026 at 12:20:01 UTC from IEEE Xplore. Restrictions apply.

the task further down to other objects to perform each sub-function. For example: When a charging session is simulated, the object *StartChargingLogic* will delegate the task of updating the status of a connector to the *ChargingStationStatusLogic* object, and it will also delegate the task of sending the message to the *OcppMessageHandler* object which will prepare the message and send it to central management system. From the perspective outside of the *StartChargingLogic* object, it will handle all the details of "StartCharging" as mentioned in OCPP. It creates an abstraction of each functionality by hiding the details of the functionality implementation from the simulator user.

C. Event driven

An event-driven programming paradigm [6] is also used along with an object-oriented programming paradigm [4]. In event-driven programming, the events drive the flow of the program. This approach will help debug problems in central management system and the simulator step by step. While simulating the virtual charging stations, three possible events will trigger the flow of the program:

- When a message is received by virtual charging station from central management system.
- When an action is performed at virtual charging station.
- When internal clock of virtual charging station reaches particular time value (i.e. When time to send new meter readings value to central management system, when time to adjust max amperes at connectors, etc.)

D. Flexibility to use the simulator as an application or as an API

Although the simulator works as a UI application, the software architecture was designed so that it can be used as a library. Offering the simulator as a library can open new possibilities where other developers can build applications on top of this simulator—further details on different ways to use OCSS as a library are discussed further in section V.

E. Platform independent

This simulator is built using an open java development kit (OpenJDK) [7]; it allows users of the simulator to run the software on any operating system if they have Java installed on their system. The decision to develop the application using Java is because both Matlab and AnyLogic software support integration with them through Java code [8], [9].

III. Software architecture

Figure 1 shows the entire software architecture of OCSS. This architecture of OCSS contains three layers:

A. Data layer

This layer of architecture contains only two objects to manage the data of a charging station. *ChargeStationInternalData* is an object that holds all non-stateful information about a charging station. For example, the number of connectors, meter readings, current amperes at each connector, max amperes

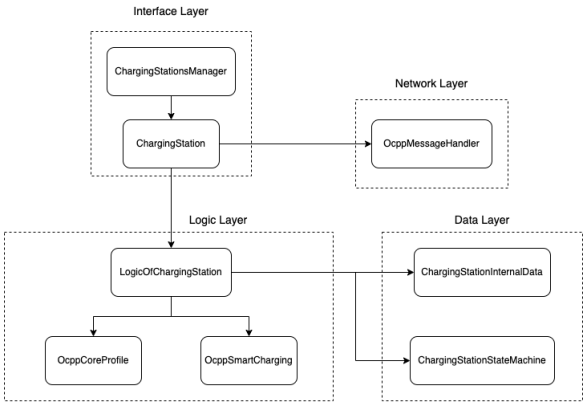


Fig. 1. Software architecture of OCSS

at each connector, internal clock, serial number and model name of the charging station are all non-stateful information. This object is simply a container for the data and performs no other functionality.

The other object in this layer is *ChargingStationStateManager*. It uses a state machine [10] to hold stateful information about the charging station. Some stateful information about the charging station is the status of the boot, availability of charging station, connection to central management status, the status of each connector, availability of each connector, status of charging at each connector and authentication user's status currently charging their EV at charging station. This object also performs sanity checks for status changes. For example, it is not possible to change the status of the connector from Available to Charging if the status of the charging station is power off mode.

B. Network layer

OcppMessageHandler is the only object in this layer. The functionality of this object is to:

- Ensure that a virtual charging station is connected to a central management system.
- Notify the *ChargingStationStateMachine* object when there is any change in connectivity status.
- Properly format and send messages to a central management system that adheres to OCPP 1.6 protocol.
- It also receives messages from a central management system and checks whether it adheres to the OCPP 1.6 protocol. It parses the message and only sends relevant information to the *ChargingStation* object.

C. Logic layer

The objects in this layer are organized based on the profiles described in OCPP 1.6 documentation. In OCPP 1.6, there are six profiles, i.e. Core, firmware management, local auth list management, reservation, smart charging and remote trigger. Each profile contains a description of several functionalities that make up that profile. This paper will discuss only the Core and smart charging profiles that we have implemented in our simulator.

Authorized licensed use limited to: FH Salzburg. Downloaded on September 29,2026 at 12:20:01 UTC from IEEE Xplore. Restrictions apply.

An object named *OcppCoreProfile* encapsulates all the functionalities described in core profile of OCPP 1.6. Any message from the central management system related to the core profile or any action performed on the virtual charging station related to the core profile will be forwarded to this object for further processing. Similar to the *OcppCoreProfile* object, *OcppSmartCharging* is responsible for handling any requests from the central management system related to smart charging/load balancing.

Both *OcppCoreProfile* and *OcppSmartCharging* themselves have other objects that represent a single functionality of the charging station as described in OCPP 1.6 working under them. Even though these objects reflect the charging station's functionality, they follow the same logic template. Following are the steps that each functionality object follows:

- Check if the message or action is related to the object. If the message or action is related to the object, it will continue to the next step.
- From the data of *ChargingStationStateMachine*, it will perform a series of checks on whether the charging station is in the right state to perform the action. If the charging station's state is valid, it will go to the next step.
- Change the data in *ChargingStationInternalData* and change the status in *ChargingStationStateMachine* according to the object's functionality.
- If the object was called due to the central management system's request message, it will construct a response and send it to *OcppMessageHandler*. If this object was called due to the action performed on the charging station, it will construct a new request message and send it to *OcppMessageHandler*.

Finally, *LogicOfChargingStation* is the root object in this layer that represents and encapsulates all the logic of a charging station. It can be viewed as the brain of the charging station. Both *OcppCoreProfile*, *OcppSmartCharging*, *ChargingStationInternalData* and *ChargingStationStateMachine* objects are present inside the *LogicOfChargingStation* object.

D. Interface layer

Users of the OCSS interact with the simulator at this layer. There are only two objects in this layer. Firstly, the *ChargingStation* object has *powerOn()*, *powerOff()*, *plugInConnector()*, *unplugConnector*, *authorizeUser()*, *startCharging()* and *stopCharging()* functions offerred to the users of the simulator. These functions reflect real life usage of charging station by electric vehicle users. This object contains both *LogicOfChargingStation* and *OcppMessageHandler* objects inside it. It forwards all the calls made to the above mentioned functions to *LogicOfChargingStation* object.

Finally, *ChargingStationsManager* is the root of all the objects in this architecture. It is considered the starting point of OCSS API. It offers users to create to start, pause and stop simulation by offering *startSimulation()*, *pauseSimulation()*, *stopSimulation()*. It also offers the users of the simulator to create and delete charging stations through the *createChargingStation()* and *deleteChargingStation* functions. This object contains a list of *ChargingStation* objects that the simulator user creates.

IV. Results

Implementing the above-proposed software architecture has led to a UI application, as shown in figure 3. For testing, we have used a regular pc to run our simulator and used Siemens ECarOC [11] as a central management system. As a result, we simulated multiple charging stations simultaneously. In figure 2, the status of a charging station is monitored thanks to the virtual charging stations simulated in the OCSS. From the perspective of the central management system, i.e. Siemens ECarOC, it cannot distinguish that they are a simulated charging station and communicate with them as if they are real charging stations.

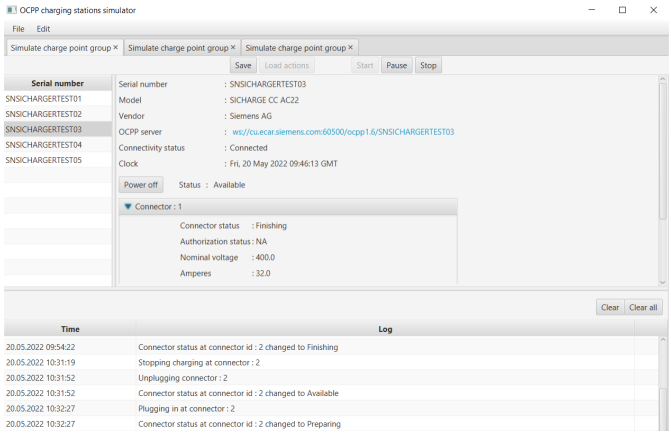


Fig. 2. UI of the simulator simulating some charging stations

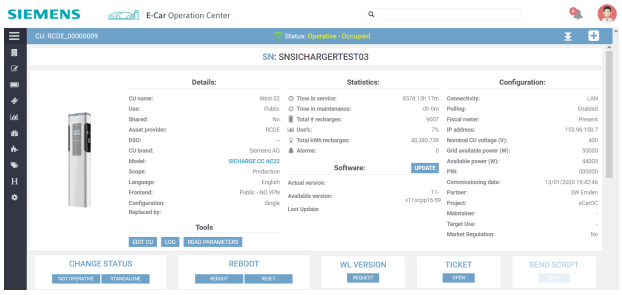


Fig. 3. Dashboard of central management system showing that one of virtual charging station is active

We were also able to test and verify the load balancing algorithm of the ECarOC central management system by simulating multiple load scenarios in the simulator without writing any additional lines of code for testing. The result of that simulation can be observed in figure 4.

V. Application

A. Development of central management system

As you can see from figures 3, 2 and 4, testing the load management was possible without writing any test code at the

Authorized licensed use limited to: FH Salzburg. Downloaded on September 29,2026 at 12:20:01 UTC from IEEE Xplore. Restrictions apply.

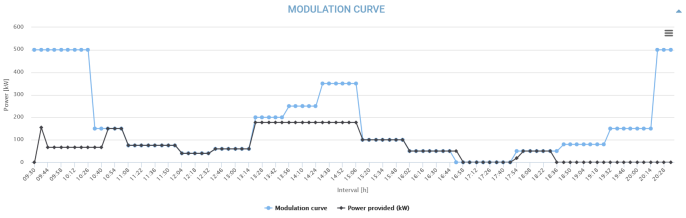


Fig. 4. Blue curve represents the desired max load curve and black line represents the actual load consumed by virtual charging stations during simulation

central management system's server. We can perform several stress tests on the central management system to see how well it can handle or recover from such test cases without writing additional test code. Some stress tests that can be performed are (not an exhaustive list):

- We could simulate how well the central management system can handle when all the charging stations are fully occupied.
- When one or more charging stations are offline from the internet network, how will the central management system reevaluate the previously calculated load profiles.
- How will the central management system respond if it receives an illogical sequence of messages from charging stations. For example, A user is not authorized to charge, but the charging station sends a message stating that a charging session has started.

B. Testing new algorithms for load balancing

Optimizing load balancing is an active research field. Researchers create algorithms that might improve the load balancing scenario. However, before deploying these algorithms, they need to be tested extensively. Although the algorithms might perform well in an isolated test environment, it is ideal to test them as close to the actual production environment. For example, figure 5 would be the typical case where the researcher has created an algorithm that can send load curve curves to the charging stations based on the total load on the grid. This scenario might work, but it isn't trivial to see how well the algorithm works with charging stations since it works directly with actual charging stations.

Figure 6 is the solution that OCSS solves by replacing real charging stations with virtual ones. In this case, it is possible to test multiple charging session scenarios and observe how the algorithm works in an actual production environment. Once the algorithm's reliability is tested, virtual charging stations can be replaced with real ones as a plug and play.

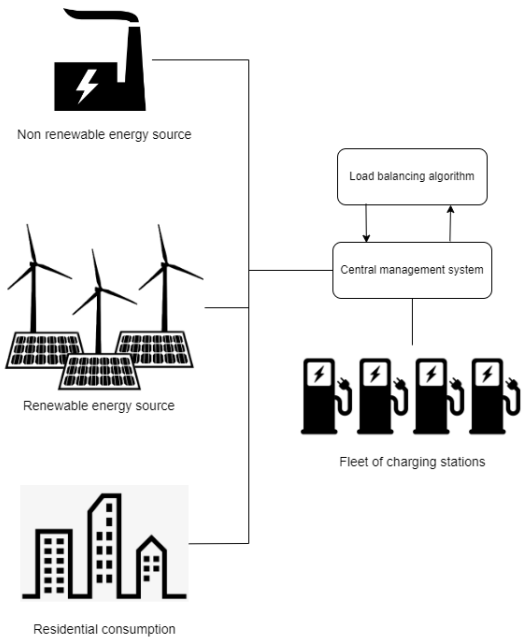


Fig. 5. Without a tool like OCSS, working of load balancing algorithm in production environment

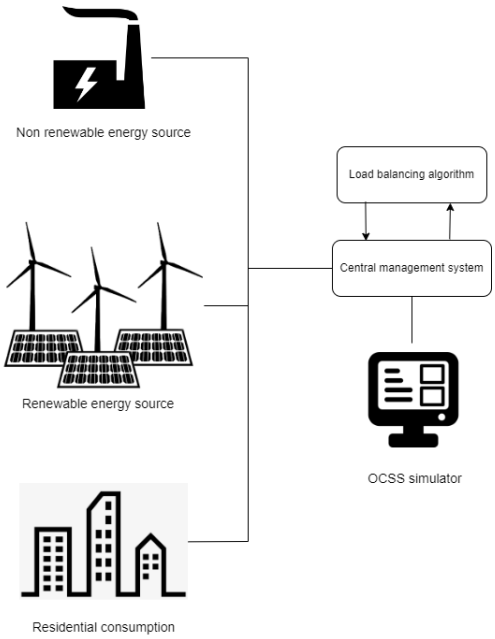


Fig. 6. With OCSS, working of load balancing algorithm can be tested directly at production environment

REFERENCES

[1] OCPP Forum, Open Charge Point Protocol, [Online] available at: https://www.openchargealliance.org/, accessed on 15.05.2022.

[2] Á. Rodríguez-Serrano, A. Torralba, E. Rodríguez-Valencia and J. Tarifa-Galisteo, "A communication system from EV to EV Service Provider based on OCPP over a wireless network," IECON 2013 - 39th Annual Conference of the IEEE Industrial Electronics Society, 2013, pp. 5434-5438, doi: 10.1109/IECON.2013.6700020

Authorized licensed use limited to: FH Salzburg. Downloaded on September 29,2026 at 12:20:01 UTC from IEEE Xplore. Restrictions apply.

[3] Ould, Martyn A., and Charles Unwin, eds. Testing in software development. Cambridge University Press, 1986, p71.
[4] Stroustrup, Bjarne. ”What is object-oriented programming?.” IEEE software 5.3 (1988): 10-20.
[5] Schmutzler, Jens, Claus Amtrup Andersen, and Christian Wietfeld. ”Evaluation of OCPP and IEC 61850 for smart charging electric vehicles.” World Electric Vehicle Journal 6.4 (2013): 863-874.
[6] Dabek, Frank, et al. ”Event-driven programming for robust software.” Proceedings of the 10th workshop on ACM SIGOPS European workshop. 2002.
[7] OpenJDK, [Online] available at: https://openjdk.java.net/ , accessed on 15.05.2022.
[8] MATLAB, Java with MATLAB - MATLAB & SIMULINK, [Online] available at: https://www.mathworks.com/help/matlab/java-language.html?s_tid=CRUX_lftnav , accessed on 15.05.2022.
[9] AnyLogic, Java in AnyLogic — AnyLogic Help. [Online] available at: https://anylogic.help/advanced/code/general.html , accessed on 15.05.2022.
[10] Seidl, M., Scholz, M., Huemer, C., & Kappel, G. (2015). UML@ classroom. Springer. pp 85 - 106.
[11] Siemens, OCWEB 2.0, [Online] available at: https://oc.ecar.siemens.com/OCWeb2/#/login , accessed on 15.05.2022.

Authorized licensed use limited to: FH Salzburg. Downloaded on September 29,2026 at 12:20:01 UTC from IEEE Xplore. Restrictions apply.